

CLAUDE.md — value-bot

Instruktionen für Claude Code in diesem Repo. **Lies diese Datei vor jeder Änderung.**

Was dieser Bot ist (und was nicht)

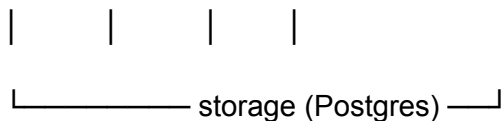
Ein **Value-Detection-Bot** für Sportwetten. Er sammelt Quoten bei **yeet.com** (dezentraler Buchmacher), vergleicht sie gegen eine **faire Linie** (Pinnacle via The Odds API, optional KI-adjustiert durch News/Lineups) und sendet **Value-Signale ab Quote 1.5** per **Telegram** — inkl. Kombiwetten.

Eiserne Regeln — niemals brechen:

- Der Bot **wettet nicht**, platziert **keine** Orders, bewegt **kein Geld**, verwaltet **keine** Wallet-Keys mit Schreibrechten. Nur **lesen** und **Signale senden**.
 - Der Bot empfiehlt **keine** Einsätze, Stakes, Kelly-Größen oder Bankroll. Nur den Value selbst (Pick, Quote, faire Quote, EV %, Edge, Grund).
 - Ein Signal entsteht nur, wenn **beide** Filter erfüllt sind: **Quote ≥ 1.5 UND EV $\geq$ Schwelle**. Niemals nur eins davon.
 - Keine "99 %"-Fantasieprozente. Ausgegeben werden echte **EV %** und **Edge**.
-

Architektur (Datenfluss)

sources → normalize → engine → alerts



scheduler treibt alles

1. **sources/** — Daten holen. Jede Quelle implementiert `OddsSource.fetch()` -> `list[OddsRecord]`.
 - `sources/the_odds_api/` — Sharp-Linie (Pinnacle). Nur die für die faire Linie nötigen Märkte abfragen (Request-Limit schonen).

- `sources/yeet/` — Ziel-Quoten von yeet.com. Drei Zugänge, in dieser Reihenfolge bevorzugen: (1) On-Chain/Subgraph, (2) Frontend-JSON-API, (3) Playwright-Scraper.
 - 2. **normalize/** — Teams/Events/Märkte vereinheitlichen. Kanonische IDs, Alias-Mapping, Fuzzy-Match (`rapidfuzz`) + LLM-Fallback. **Hier entsteht Phantom-Value bei Schlamperei — sauber arbeiten.**
 - 3. **engine/** — die Mathematik:
 - `novig.py` — Marge aus Sharp-Quote rausrechnen → `p_fair`.
 - `ki_adjust.py` — News/Lineup-Delta via Claude API, **gekappt** (max ± 0.06).
 - `value.py` — EV, Edge, Filter.
 - `parlay.py` — Kombis aus +EV-Legs.
 - 4. **alerts/** — Telegram. `telegram.py` (Bot/Befehle), `formatting.py` (Nachricht).
 - 5. **storage/** — SQLAlchemy-Modelle, Alembic-Migrationen. Quoten als Zeitreihe (für CLV).
 - 6. **scheduler/** — APScheduler-Jobs, Polling-Intervalle, Rate-Limits.
 - 7. **backtest/** — Engine gegen historische Odds zur Filter-Kalibrierung.
-

Die Kern-Mathematik (nicht abweichen)

Faire Wahrscheinlichkeit aus Sharp-Quote (No-Vig):

$p_i = 1 / \text{odds}_i$ # je Ausgang

$M = \sum p_i$ # Overround (Marge)

$p_{\text{fair}_i} = p_i / M$

KI-Adjustierung (begrenzt):

$p_{\text{true}} = \text{clip}(p_{\text{fair}} + \text{delta_ki}, 0.01, 0.99)$ # $\text{delta_ki} \in [-0.06, +0.06]$

Wenn die Sharp-Linie sich schon bewegt hat → die Info ist eingepreist → `delta_ki` gegen 0. Doppelzählen vermeiden.

Value (gegen yeet.com-Quote):

$\text{EV} = p_{\text{true}} * \text{odds_yeet} - 1$

$\text{Edge} = p_{\text{true}} - (1 / \text{odds_yeet})$

Signal-Filter (alle müssen zutreffen, Schwellen in `config/`):

`odds_yeet >= 1.5`

`EV >= 0.03`

`Edge >= 0.02`

Markt hat eine Sharp-Referenz in The Odds API

KI-Confidence >= Schwelle (sonst Signal-Typ = "info")

Kombis (parlay.py): nur aus Legs, die einzeln den Filter bestehen; möglichst unabhängig (keine zwei Legs aus demselben Spiel); `odds_combo = Π odds_i`, `p_combo = Π p_true_i`, `EV_combo = p_combo * odds_combo - 1`; höhere Schwelle (`EV_combo ≥ 0.08`); max. 2–4 Legs.

Tech-Stack

Python 3.12 · `asyncio` + `APScheduler` · `httpx` · `Playwright` (nur falls yeet gescraped wird) · PostgreSQL + SQLAlchemy + Alembic · Redis (Dedup, Rate-Limit-Token) · `python-telegram-bot` · `anthropic` (KI-Adjustierung) · Docker + docker-compose.

Befehle

Setup

`docker-compose up -d` # postgres, redis, app, scheduler

`pip install -r requirements.txt`

`alembic upgrade head` # DB-Migrationen

Entwicklung

`pytest` # alle Tests — MUSS grün sein vor Commit

`pytest tests/engine/ -v` # gezielt

`ruff check . && ruff format .` # Lint + Format

```
mypy .                # Typecheck

# Lokaler Lauf

python -m scheduler.jobs    # Polling-Loops starten

python -m backtest.run --sport soccer --from 2025-01-01
```

Wenn diese Befehle/Tools noch nicht existieren, lege sie beim ersten Mal an (requirements, ruff/mypy-Config, pytest-Setup) und aktualisiere diesen Abschnitt.

Konventionen

- **Async durchgängig** für I/O (HTTP, DB, Telegram). Keine blockierenden Calls im Event-Loop.
 - **Type Hints überall**, `mypy` muss durchlaufen.
 - **Eine Verantwortung pro Modul**. Die Engine kennt keine Quellen-Details; Quellen kennen keine Engine. Kopplung nur über `OddsRecord`/DB.
 - **Konfiguration in `config/`** (Pydantic-Settings, aus `.env`). Keine Magic Numbers im Code — Schwellen (1.5, 0.03, ± 0.06 ...) sind konfigurierbar.
 - **Geldwerte/Quoten als `Decimal`**, nicht `float`.
 - **Jede Quote mit Timestamp speichern** — Zeitreihe ist Pflicht für CLV.
 - **Fehler isolieren**: Bricht eine Quelle/ein Scraper, darf der Rest weiterlaufen. Try/except pro Quelle, Logging, weiter.
 - **Idempotente Alerts**: Dedup über Redis. Gleiches Signal nicht doppelt senden; nur bei *neuem* oder *verbessertem* Value re-alerten.
-

Secrets & Sicherheit

- **Nie** Secrets committen. Alles in `.env` (gitignored): `THE_ODDS_API_KEY`, `TELEGRAM_BOT_TOKEN`, `ANTHROPIC_API_KEY`, ggf. `YEET_RPC_URL/YEET_SUBGRAPH_URL`, DB/Redis-URLs.
 - **Read-only gegenüber yeet.com**. Kein Private-Key mit Signier-/Schreibrechten im Repo oder zur Laufzeit. On-Chain nur lesen.
 - `.gitignore` strikt halten (`.env`, `__pycache__`, Playwright-Profile, Logs).
-

Tests & "Definition of Done"

Vor jedem PR:

1. `pytest` grün — besonders `tests/engine/` (No-Vig, EV, Filter, Parlay) mit festen Beispielwerten.
2. `ruff check`, `ruff format --check`, `mypy` ohne Fehler.
3. Neue Logik → neuer Test. Engine-Mathe immer mit konkreten Zahlenbeispielen testen (z.B. Pinnacle 1.80/2.10 → `p_fair` 0.538).
4. Bei Schema-Änderung → Alembic-Migration dabei.
5. Keine Quelle/kein Scraper hart verdrahtet — über das `OddsSource`-Interface.

Niemals als fertig markieren, wenn: Tests rot, Migration fehlt, Secrets im Diff, oder ein Filter umgangen wird.

Bau-Reihenfolge (eine Phase pro PR)

0. **Fundament** — Repo, Docker, Postgres, Redis, CI, dieses CLAUDE.md. → `docker-compose up` läuft, Tests grün.
1. **yeet.com erschließen** — On-Chain/Subgraph/Frontend prüfen, Quoten normalisiert in DB.
2. **Sharp-Linie** — The Odds API (Pinnacle), Event-Matching yeet↔Sharp.
3. **Engine + Filter** — No-Vig, EV, Quote-≥1.5-Filter. Erste Value-Liste in Logs.
4. **Telegram** — Bot, Format, Dedup, Befehle (`/value`, `/sport`, `/minev`, `/stop`).
5. **KI-Adjustierung** — News/Lineup-Ingestion, Claude-Delta (gekappt), Confidence-Gate.
6. **Parlay-Builder** — Kombis aus +EV-Legs.
7. **Alle Sportarten** — ausrollen.
8. **Tracking & Backtest** — CLV, Trefferquote, wöchentlicher Report per Telegram.

Erst eine Sportart end-to-end fertig bauen (Phasen 1–6), dann ausrollen.

Wenn unklar

Lieber **fragen** / **klein halten** / **testen** als raten. Bei Quellen-Eigenheiten (z.B. yeet.com-Datenformat) erst die Datenquelle inspizieren und das tatsächliche Antwort-Schema dokumentieren, dann den Parser darauf bauen — nicht auf Annahmen.